

Homework 4 — AI Harness Systems Design and Analysis (Syllabus Version)

設計內容須包含：

- AI 應用問題定義與使用情境
- AI system architecture (LLM + tools + memory)
- function calling / tool usage 機制說明
- 至少 3 個工具 (API / function) 設計
- agent workflow (多步驟任務執行流程)
- evaluation 方法 (如何衡量系統效果)
- AI orchestration (流程控制與決策方式)

繳交項目 (Deliverables)

1. 書面報告 (必交)

- 2-5 頁 (A4 或 IEEE 格式皆可)
- 內容須涵蓋：
 - 問題定義與應用背景
 - AI Harness 系統設計
 - tools 設計 (至少 3 個)
 - workflow / agent 流程說明
 - evaluation 方法

2. 資訊圖表 (Infographic , 必交)

需以視覺化方式呈現系統設計，內容包含：

- AI system architecture (LLM 、 tools 、 memory)
- orchestration / workflow flow
- function calling 或 tool chain 流程
- 可包含 sequence diagram 或 pipeline 視覺化

3. log.md (必交)

需記錄 AI 輔助設計與開發過程，包括：

- 與 AI 的互動紀錄（prompt / chat history）
- 系統設計迭代過程
- 架構調整與設計決策
- 問題分析與修正過程

這是一份非常標準且結構完整的 AI 系統設計作業！這類型的作業核心在於**系統架構思維（System Architecture）**，也就是如何把 LLM 當成「大腦」（Controller），並透過規劃好的管道（Harness）讓它去驅動各種工具、記憶體和工作流。

為了幫你拿到高分，我們可以把這份作業拆解成一條清晰的開發與撰寫路徑。以下我為你整理的設計框架與核心攻略：

💡 推薦選題策略（具備高豐富度、好設計工具的場景）

建議選擇資料流豐富、且需要多步驟推理（Multi-hop reasoning）的場景，這樣你的流程圖（Infographic）和工具設計才会有足夠的深度。

- 推薦場景一：智能馬拉松訓練與健康管理助理（個人化教練代理）
 - 為什麼好寫？ LLM 需要根據用戶的賽事目標（如：半馬/全馬目標時間、當前週跑量）、當日身體數據與天氣，動態調整訓練課表。
 - 工具設計：天氣 API、穿戴裝置數據讀取 API、課表資料庫（RAG）。
- 推薦場景二：智能理財與自動化資產配置助理
 - 工具設計：即時股價 API、用戶資產倉位查詢 API、風險評估計算機。

🔧 三大繳交項目（Deliverables）撰寫與設計攻略

1. 書面報告（2-5 頁）：核心系統架構

在報告中，不要只是平鋪直敘地寫代碼，要用**系統設計的角度**來描述。以下是建議的四大章節架構：

一、問題定義與使用情境（Problem Definition）

- 明確定義輸入（**Input**）與預期輸出（**Expected Output**）。
- 舉出一個具體的使用者情境（**User Persona & User Story**）。例如：「使用者想要知道在明天下雨、且週跑量已達 50 公里的情況下，應該進行什麼強度的間歇跑。」

二、 AI Harness 系統架構設計（**System Architecture**）

- 大腦（**Controller**）：選用什麼模型？（如 GPT-4o, Claude 3.5 Sonnet），其 Function Calling 的穩定度如何？
- 記憶（**Memory**）：區分 **Short-term Memory**（當前對話上下文，用於維持 Chat History）與 **Long-term Memory**（用戶的歷史偏好、過去的訓練表現數據，存於 Vector DB 或 Relational DB）。
- 工具箱（**Tools**）：提供給 LLM 的外部接口。

三、 工具設計與 **Function Calling** 機制（至少 3 個）

這是作業的技術亮點，請務必寫出精確的 **JSON Schema**。LLM 就是靠這個 Schema 來決定何時該呼叫工具。

💡 **評分關鍵**：工具的描述（description）一定要寫得非常清晰，因為 LLM 是根據描述來做語義匹配（Semantic Matching）進而觸發呼叫的。

JSON

```
{  
  
  "name": "get_weather_forecast",  
  
  "description": "取得特定城市與日期的天氣預報，用於判斷是否適合進行戶外高強度訓練。",  
  
  "parameters": {  
  
    "type": "object",  
  
    "properties": {  
  
      "location": { "type": "string", "description": "城市名稱，例如：台中市"},  
  
      "date": { "type": "string", "description": "ISO 格式日期，例如：2026-05-21"}  
    }  
  }  
}
```

```

    },
    "required": ["location", "date"]
  }
}

```

四、 流程控制與決策（Orchestration & Workflow）

- 說明你使用的是 **Linear Sequential**（線性流程）、**Router**（路由分支）還是 **Agentic Loop**（帶有反思機制的循環流程，如 **LangGraph** 結構）。
- 描述 LLM 遇到無法一步解決的問題時，如何進行「思考 -> 呼叫工具 -> 得到結果 -> 再次思考（ReAct 模式）」的決策過程。

五、 Evaluation 方法（系統評估）

- **RAG 評估**： 如果有用到知識庫，可以使用 Ragas 框架，評估 Context Relevance（檢索相關性）和 Faithfulness（忠實度）。
- **Function Calling 準確率**： 建立一個含有 20 個典型問題的測試集（Ground Truth），測試 LLM 是否能在正確的情境下呼叫正確的工具、並帶入正確的參數。

2. 資訊圖表（Infographic）：視覺化你的 Harness

這部分佔了 10%，是拉開差距的關鍵。建議使用 **Figma**、**Miro** 或 **Draw.io** 來繪製，圖表應包含兩個核心視角：

1. 靜態架構圖（Architecture Diagram）：

- 切分成三層：前端/用戶層（UI/UX） — 編排層（LLM Orchestrator + Memory） — 外部服務層（Tools/APIs/Vector DB）。

2. 動態資料流/時序圖（Sequence / Data Flow Diagram）：

- 用箭頭標示出資料的順序：
 1. 用戶輸入提問 → 2. LLM 分析並回傳 tool_calls → 3. 系統攔截並執行本地 Function → 4. Function 將結果回傳給

LLM → 5. LLM 生成最終白話文答案 → 6. 呈現給用戶。

3. log.md：設計過程的黑盒子（10% 必拿分數）

這份文件是用來向教授證明「這真的是你一步步調教、設計出來的，而不是最後一秒用 AI 拼湊的」。建議採用時間軸（Timeline）或迭代（Iteration）的格式撰寫：

- **Iteration 1：** 初步 Prompt 與工具設計。發現 LLM 在面對複合問題時，常常漏掉呼叫第二個工具（例如：查了天氣卻忘了查用戶的當前體能狀態）。
- **Decision 1：** 修改系統提示詞（System Prompt），加入 Few-shot examples（少樣本提示），強制 LLM 在回答前必須先規劃工具呼叫鏈（Tool Chain）。
- **Iteration 2：** 測試邊界條件（Edge Cases）。當 API 回傳錯誤（500 Error）或查無資料時，LLM 該如何優雅地回應，而不是直接閃退或胡言亂語（Hallucination）。

接下來，我們如何開始？

你可以先根據你的興趣或專業背景選定一個場景。如果你想以我們剛剛提到的「智能運動與健康管理助理」或是其他你感興趣的場景（例如：學術文獻分析、自動化代辦助理）為例，我們可以一起把架構和 3 個工具的詳細規格（JSON Schema）直接生出來！

你想選擇哪一個應用場景作為這份作業的主角呢？